# 机器人ROS基础与实践



# 第一章 ROS2 核心概念



# 目录


1. 初识ROS

2. ROS2 vs ROS1

3. ROS2核心概念

4. ROS2安装方法

5. ROS2体验：小海龟仿真



# 1. 初识 ROS

# 1.1 ROS的诞生


















ROS中元老机器人--PR2

# 1.1 什么是ROS?

- ROS（Robot Operating System，机器人操作系统） 是一款面向机器人的开源**元操作系统**。
- 它具备传统操作系统所能提供的各类基础服务，包含硬件抽象、底层设备控制、通用功能实现、进程间消息传输以及软件包管理等。
- ROS并非底层操作系统，而是一套面向机器人开发的分布式软件中间件与工具生态。它提供标准化节点通信框架、硬件驱动、可视化、定位建图等通用功能，把感知、控制、规划模块化解耦，开发者无需重复开发底层通信逻辑，可快速复用开源算法，适配轮式、足式、机械臂等各类机器人。
- ROS2是面向实时、多平台的新一代ROS迭代版本。

# 1.2 为什么选择ROS?

- **提高机器人研发的软件复用率**

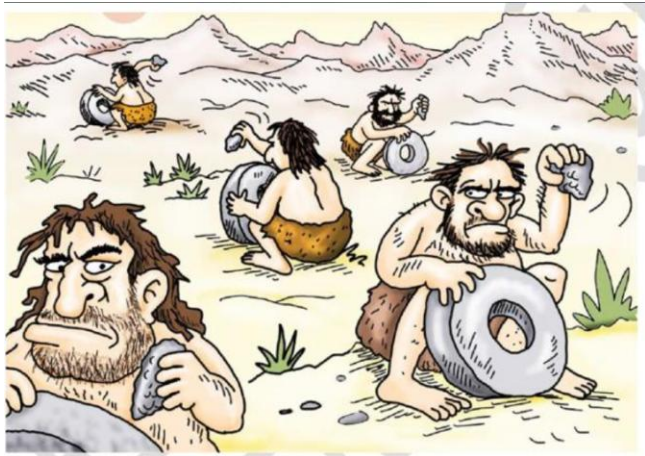
(1) 点对点的设计

(2) 多语言支持

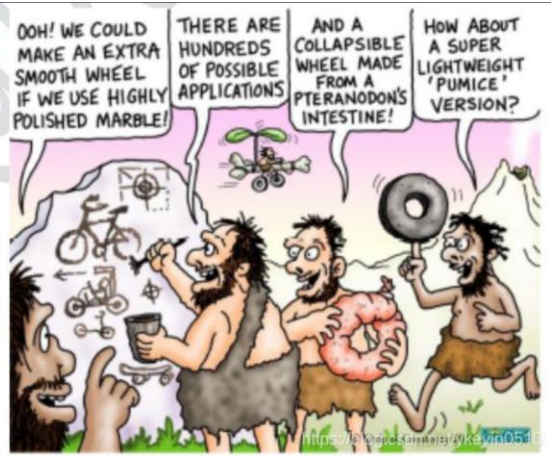
(3) 架构精简、集成度高

(4) 组件化工具包、功能包丰富

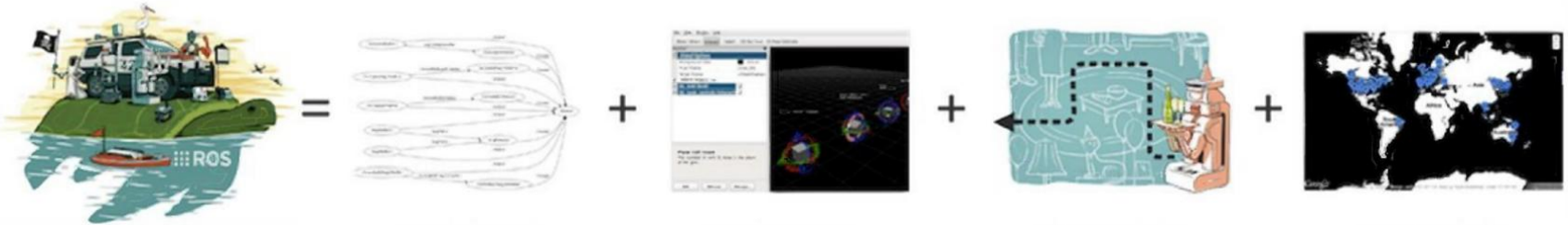
(5) 免费并且开源



传统模式




现代模式

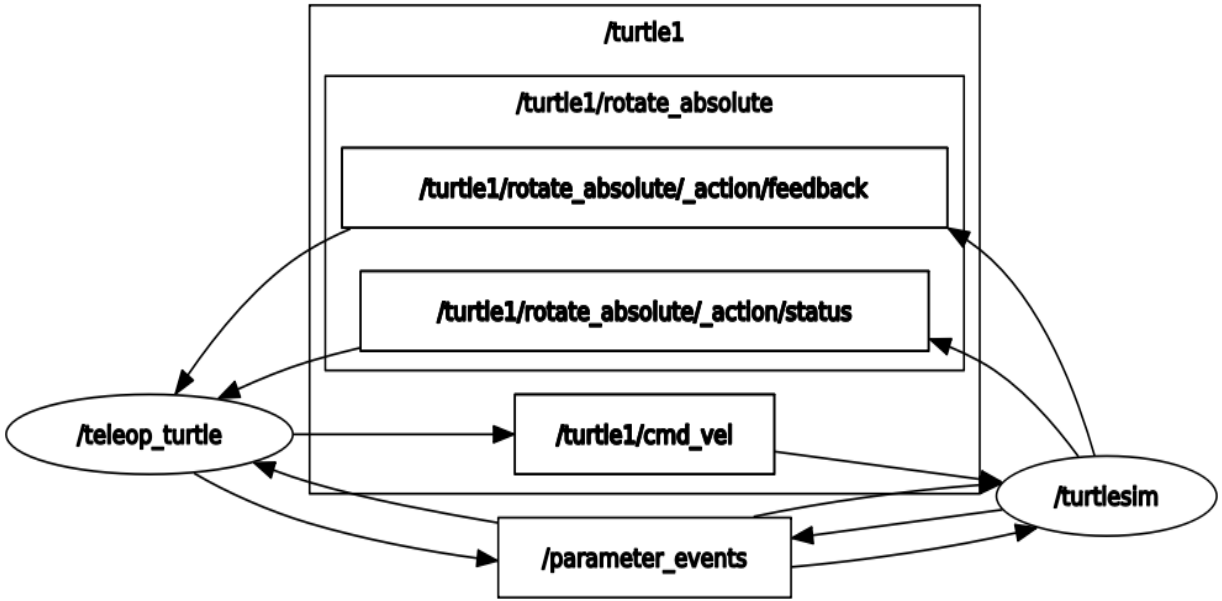



通信机制　开发工具　应用功能　生态系统

# 1.2 为什么选择ROS?

## 松耦合分布式通信

- 松耦合：节点间无直接依赖，靠标准化消息接口交互，独立启停互不干扰
- 分布式：底层基于网络协议、代码不区分本机/跨机、支持多设备局域网组网透明通信

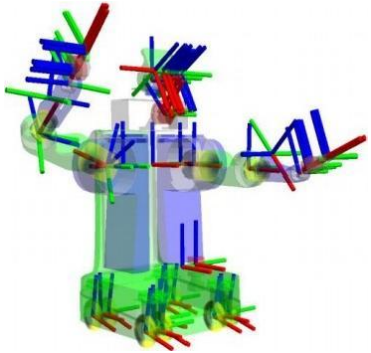


# 1.2 为什么选择ROS?

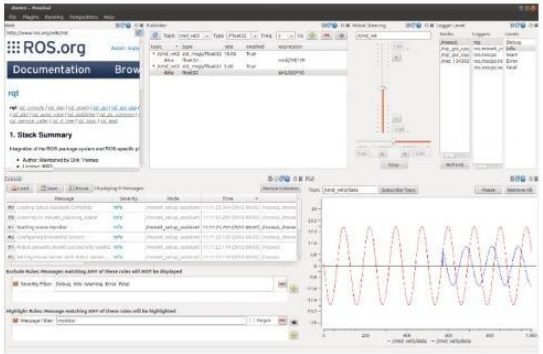
## ➢ 丰富的组件化工具包

colcon build/clean
source install/setup.bash
ros2 pkg list/prefix
ros2 run
ros2 launch
ros2 node list/info
ros2 topic list/echo/pub
ros2 service list/call
ros2 interface show
……

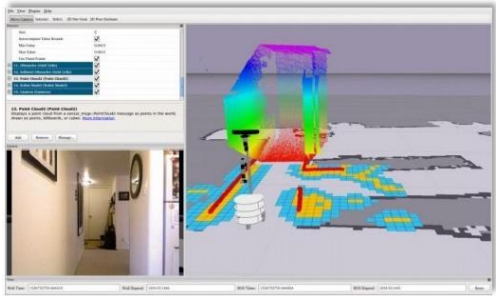
命令行&编译器



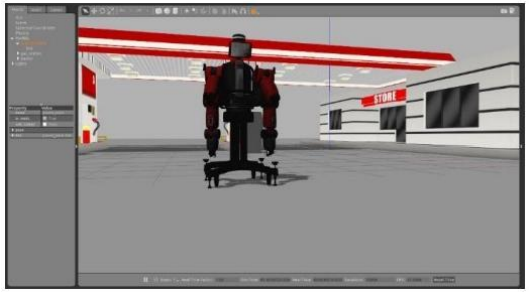
TF坐标变换




Qt 工具箱



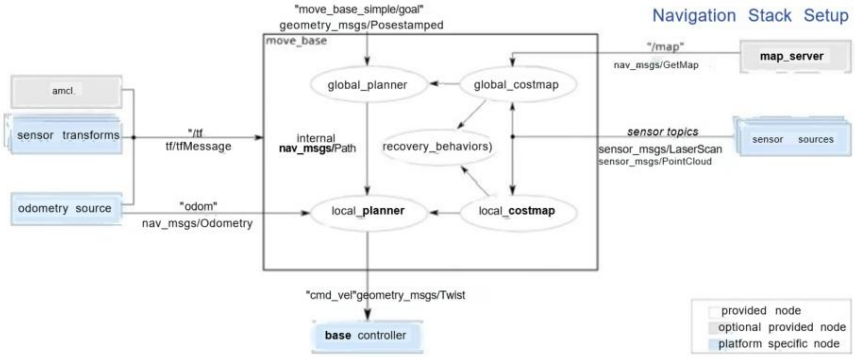
RViz



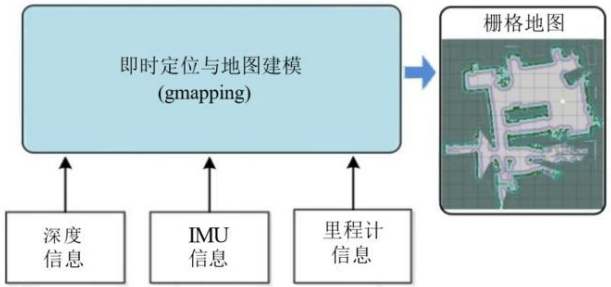

Gazebo

# 1.2 为什么选择ROS?

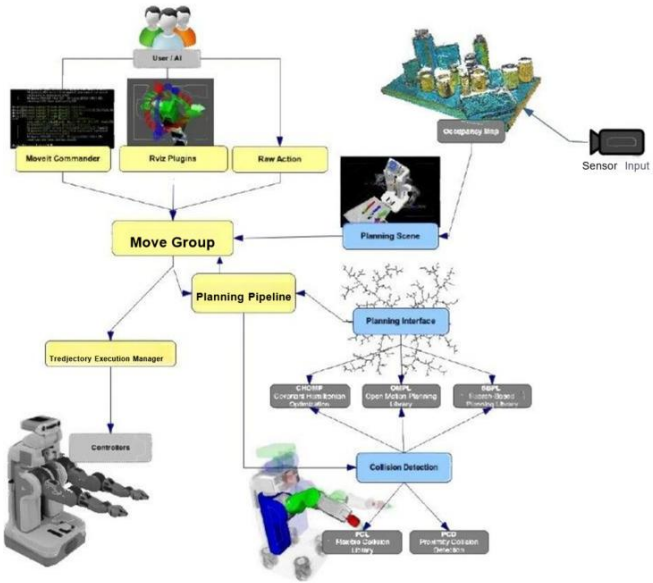
## ➢ 丰富的功能包




Navigation



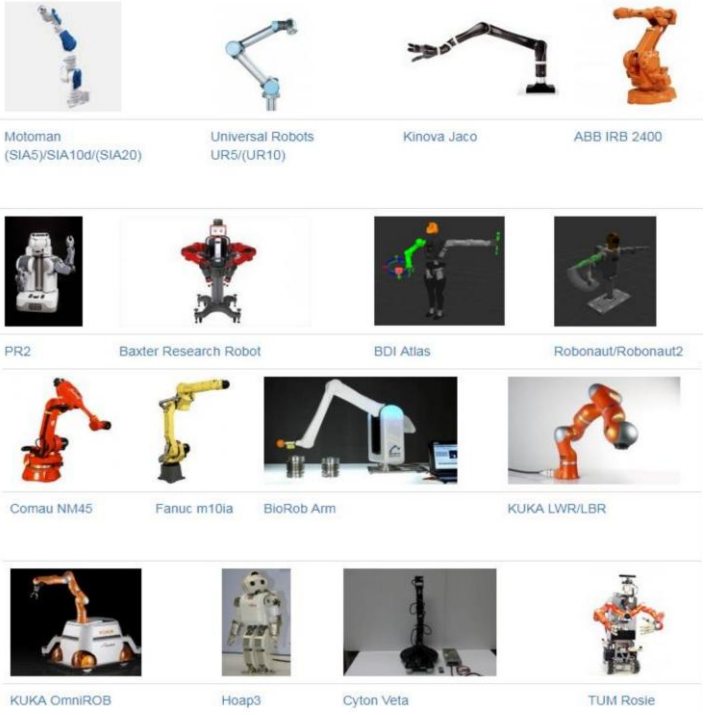

SLAM

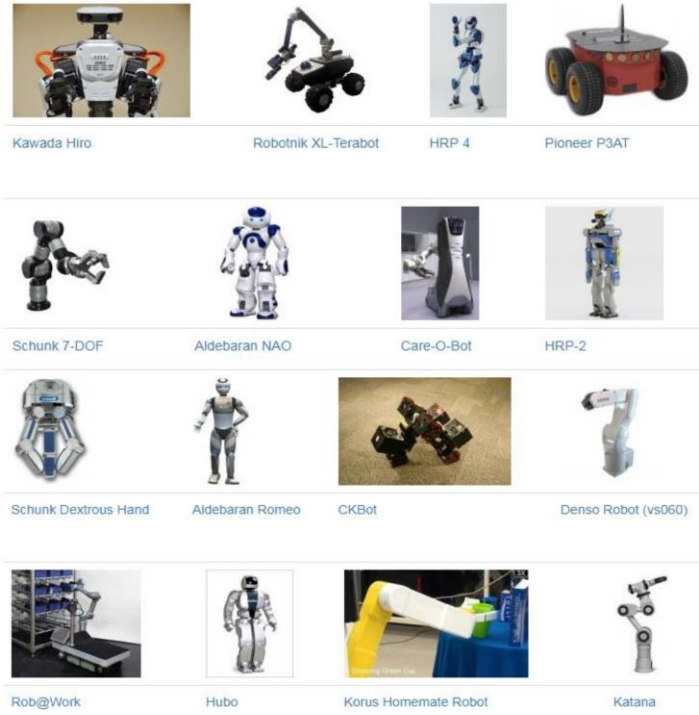



MoveIt!

# 1.2 为什么选择ROS?

- **完善的生态**

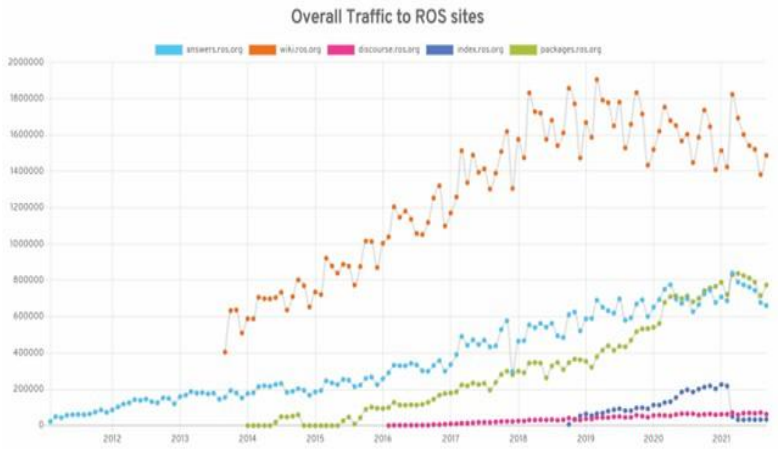


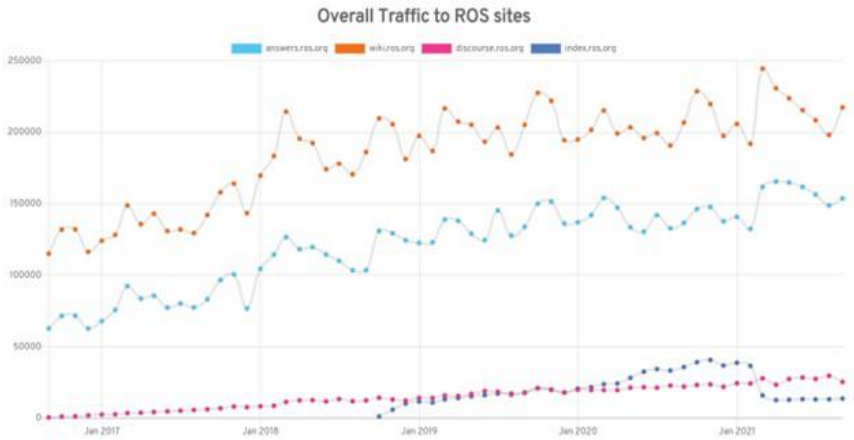




**Powering the world's robots! ——机器人领域的普遍标准**

# 1.2 为什么选择ROS?

- **ROS社区的繁荣**



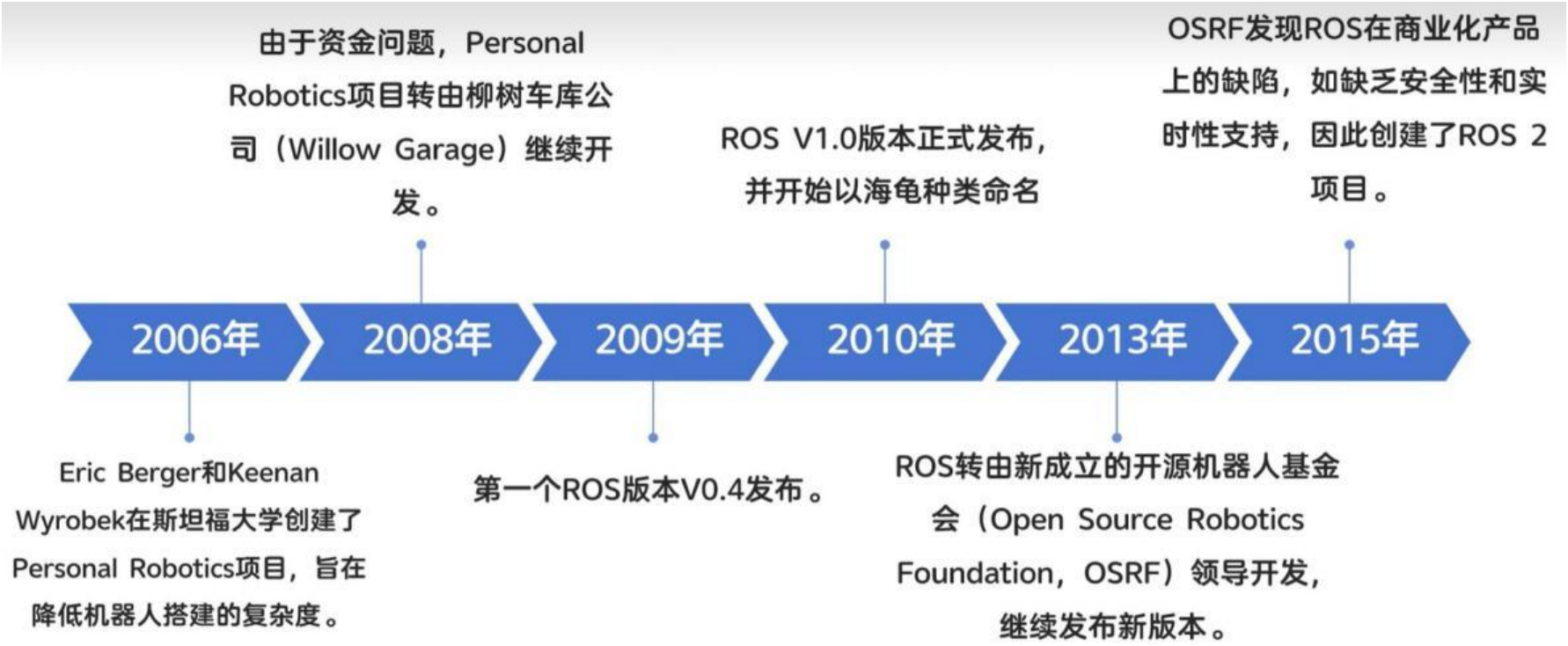




**每个月基本有250万左右的访问量，已经是一个活跃度非常棒的社区**



# 2. ROS2 vs ROS1

# 2.1 ROS的发展历程


2006年
Eric Berger和Keenan Wyrobek在斯坦福大学创建了Personal Robotics项目，旨在降低机器人搭建的复杂度。
2008年
由于资金问题，Personal Robotics项目转由柳树车库公司（Willow Garage）继续开发。
2009年
第一个ROS版本V0.4发布。
2010年
ROS V1.0版本正式发布，并开始以海龟种类命名
2013年
ROS转由新成立的开源机器人基金会（Open Source Robotics Foundation，OSRF）领导开发，继续发布新版本。
2015年
OSRF发现ROS在商业化产品上的缺陷，如缺乏安全性和实时性支持，因此创建了ROS 2项目。

# 2.1 ROS的发展历程


ROS 2的第一个五年长期支持版Humble Hawksbill正式发布，标志着ROS 2趋向成熟。
2016年
2022年5月
2022年12月
.....
OSRF成立子公司OSRC（Open Source Robotics Corporation），以在商业和盈利上进行探索。
OSRC被Google X下属的Intrinsic公司收购，而OSRF将继续作为独立的非营利组织运营

# 2.2 ROS1局限性根源









- 独立的机器人（PR2）
- 没有实时性的强需求
- 主要应用于学术研究
- 工作站级的计算资源
- 拥有良好的网络连接
- 机器人类型与领域少





# 2.2 ROS1局限性根源

- ROS的应用领域已经不局限于学术研究
- 基于ROS的产品已经逐渐走向市场
- 支持机器人的种类繁多：轮式机器人，人形机器人，工业机械手，室外机器人、飞行器，救援机器人....



# 2.2 ROS2 全新的ROS

**架构的颠覆**

- ROS1的架构下，所有节点需要使Master进行管理
- ROS2使用基于DDS的Discovery机制，Master说拜拜

**API的重新设计**

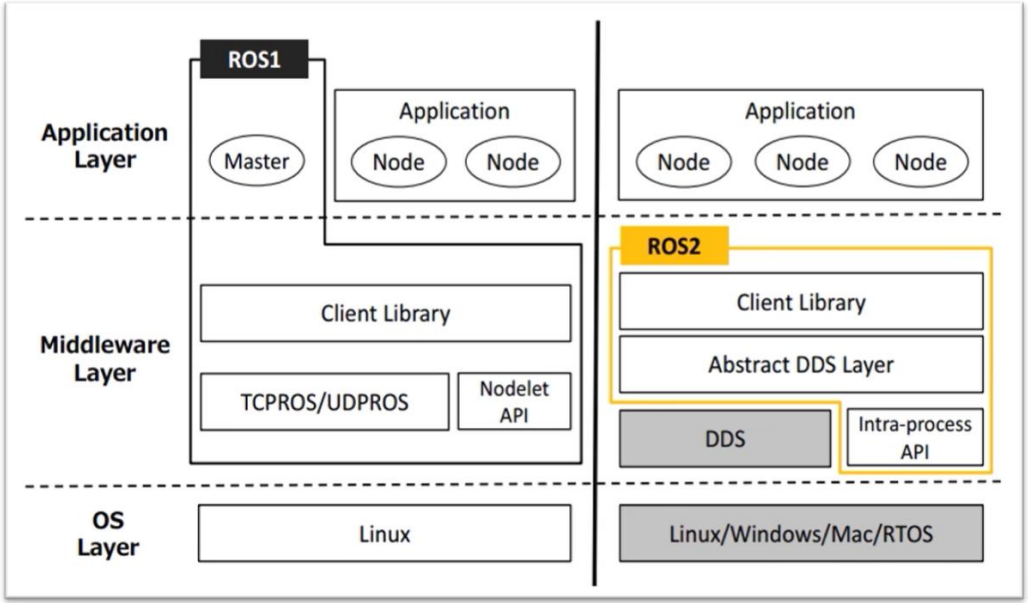
- ROS1中的大部分代码都基于2009年2月设计的API
- ROS2重新设计了用户API，但使用方法类似

**架构的颠覆**

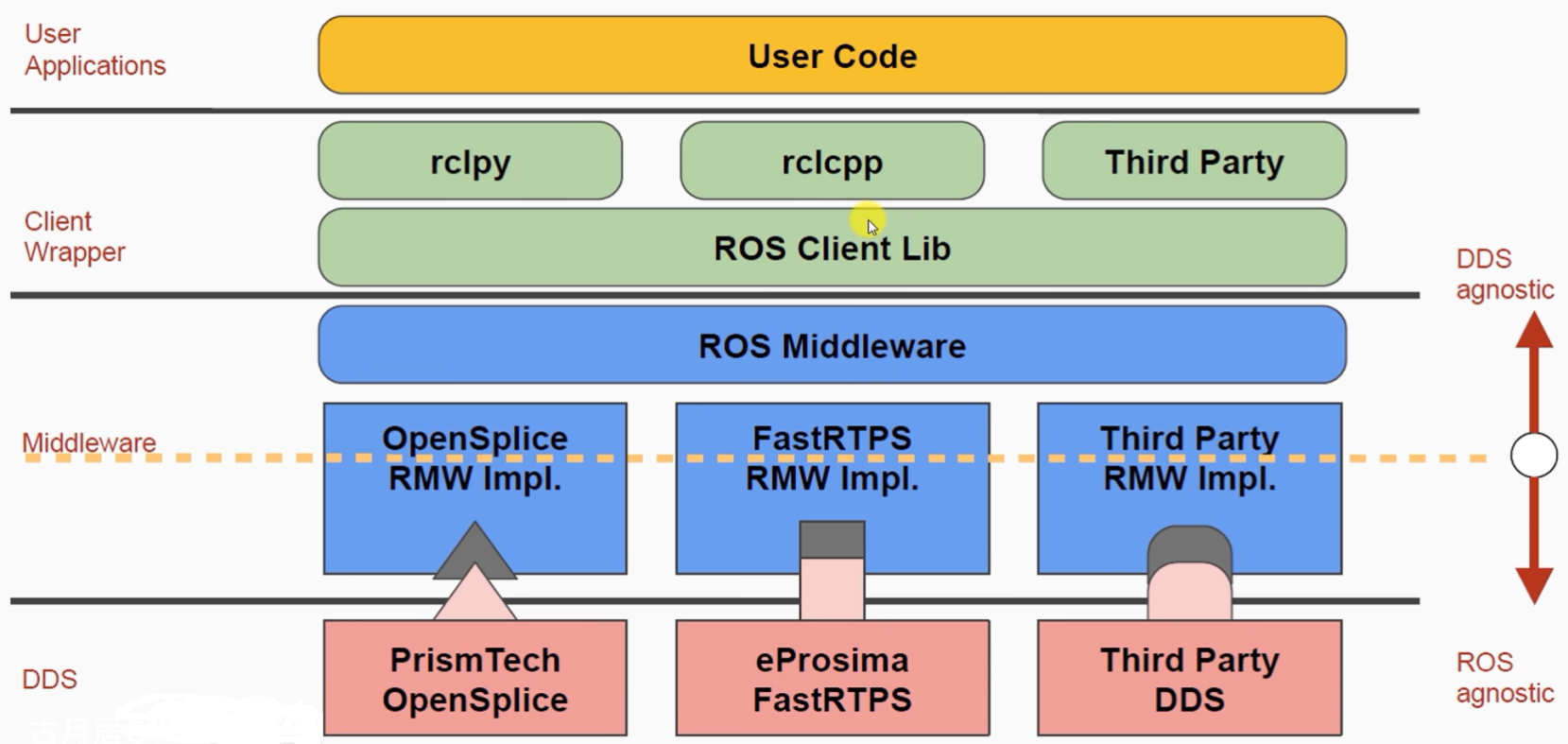
- ROS1使用rosbuild、catkin管理项目
- ROS2使用升级版的ament、colcon

# 2.3 ROS2 vs ROS1 核心差异—架构

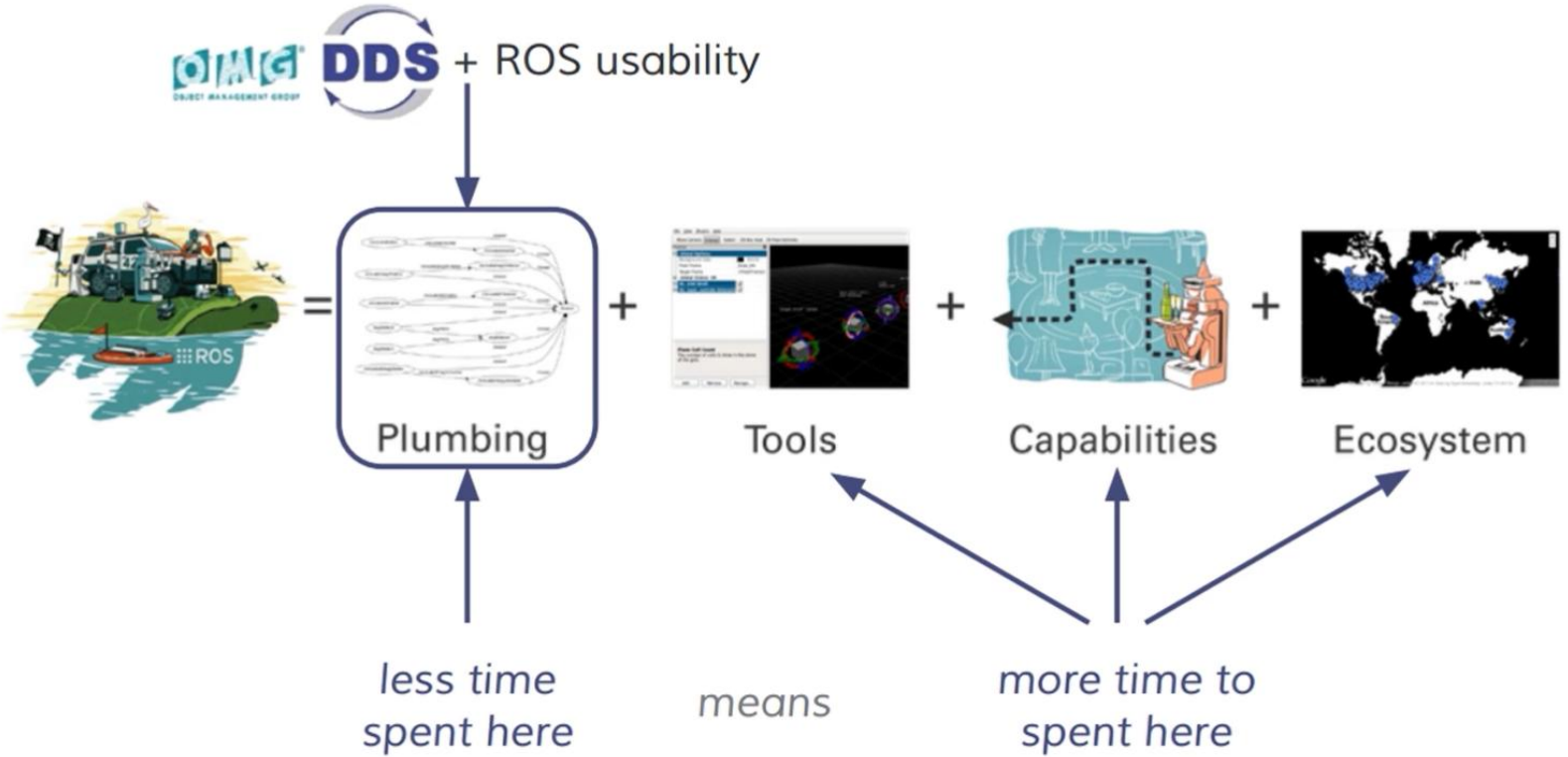
- **OS**
  - ROS1：Linux
  - ROS2：Linux、Windows、MAC、RTOS
- **通讯**
  - ROS1：TCPROS/UDPROS
  - ROS2：DDS
- **节点模型**
  - ROS1：publish/subscribe
  - ROS2：discovery
- **进程**
  - ROS1： Nodelet
  - ROS2： Intra-process



# 2.3 ROS2 vs ROS1 核心差异—DDS



# 2.3 ROS2 vs ROS1 核心差异—DDS



# 2.3 ROS2 vs ROS1 核心差异

| 对比维度 | ROS1 (Noetic) | ROS2 (Humble) |
| --- | --- | --- |
| **通信中间件** | XMLRPC + TCPROS，**中心化 Master** | DDS（Data Distribution Service），**去中心化自动发现** |
| **实时性** | 弱，无 QoS | 强，支持 QoS + 实时内核 |
| **跨平台** | Linux 为主 | Linux / Windows / macOS / RTOS |
| **安全性** | 无加密认证 | SROS2、加密、访问控制 |
| **节点管理** | **需要 roscore** | **无需 Master，自动发现** |
| **生态成熟度** | **历史包丰富** | 导航/SLAM/驱动主流已迁移 |

# 2.4 ROS2发行版本

| | | | | |
|---|---|---|---|---|
| Eloquent Elusor | November 22, 2019 |  | November 2020 | Michael Carroll |
| Dashing Diademata | May 31, 2019 |  | May 2021 | Steven! Ragnarök |
| Crystal Clemmys | December 14, 2018 |  | December 2019 | Steven! Ragnarök |
| Bouncy Bolson | July 2, 2018 |  | July 2019 | Mikael Arguedas / Steven! Ragnarök |
| Ardent Apalone | December 8, 2017 |  | December 2018 | Steven! Ragnarök |
| beta3 | September 13, 2017 | | December 2017 | |
| beta2 | July 5, 2017 | | September 2017 | |
| beta1 | December 19, 2016 | | Jul 2017 | |
| alpha1 - alpha8 | August 31, 2015 | | December 2016 | |

[Distributions — ROS 2 Documentation: Humble documentation]

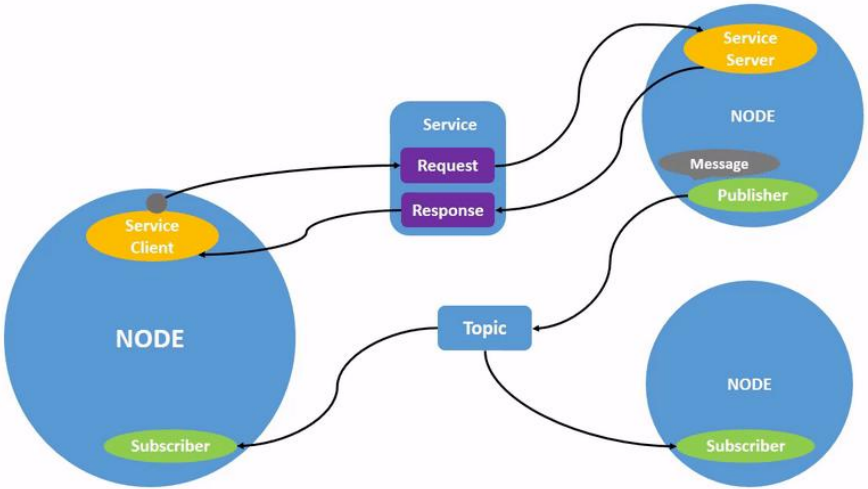
| Distro | Release date | Logo | EOL date | ROS Boss |
|---|---|---|---|---|
| Lyrical Luth | May 22, 2026 |  | May 2031 | Shane Loretz |
| Kilted Kaiju | May 23, 2025 |  | December 2026 | Scott K Logan |
| Jazzy Jalisco | May 23, 2024 |  | May 2029 | Marco A. Gutiérrez |
| Iron Irwini | May 23, 2023 |  | December 4, 2024 | Yadunund Vijay |
| Humble Hawksbill | May 23, 2022 |  | May 2027 | Christophe Bédard / Audrow Nash |
| Galactic Geochelone | May 23, 2021 |  | December 9, 2022 | Scott Logan |
| Foxy Fitzroy | June 5, 2020 |  | June 20, 2023 | Jacob Perron / Dharini Dutia |



# 3. ROS2核心概念

# 3.1 节点 (Node)

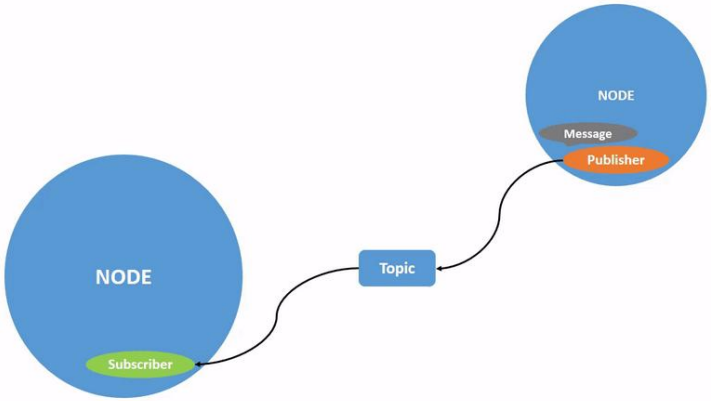
- 节点 (Node) 是机器人系统里一个独立的小功能模块
- 每个节点通常聚焦于某一个功能，比如相机驱动、激光雷达驱动、定位、导航、显示等
- 节点之间不直接调用对方，而是通过话题（Topic）、服务（Service）、动作（Action）、参数（Parameters）等互相通信
- 节点可以单独启动、单独关闭、单独调试，让机器人系统变得模块化、易维护

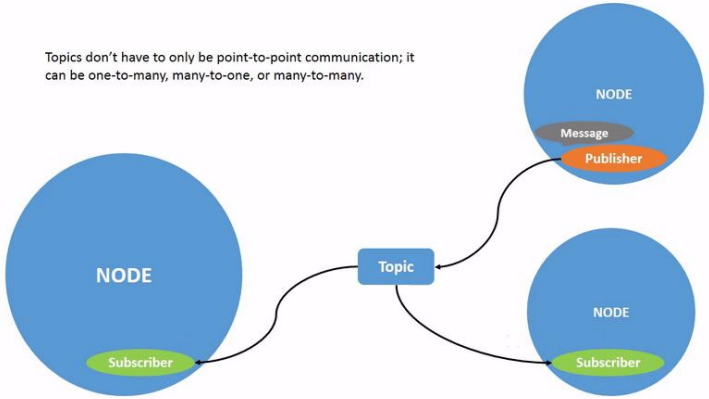



Understanding nodes — ROS 2 Documentation: Humble documentation

# 3.2 话题（Topic）

- 话题（Topic）是 ROS2 节点之间 “单向、持续、异步” 传递数据的通道
- 节点通过发布（Publish）往Topic发数据
- 其他节点通过订阅（Subscribe）从Topic收数据
- 发布者和订阅者互相不知道对方存在，解耦、灵活
- 一个Topic只能传一种固定消息类型（如 Image、LaserScan），常用于传感器数据、状态、视频流等持续数据流
- Topic可以是一对一、一对多、多对一、多对多的

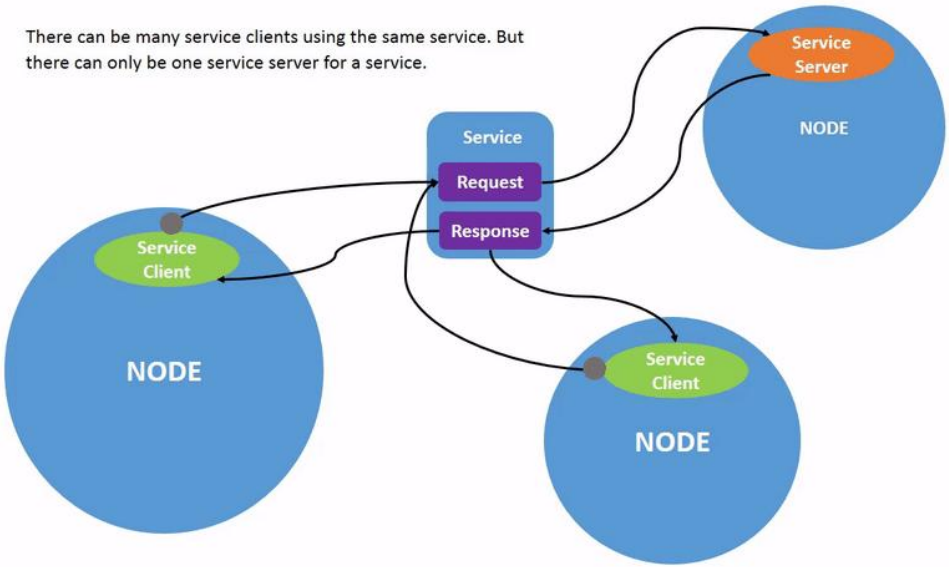



[Understanding topics — ROS 2 Documentation: Humble documentation]

# 3.3 服务 (Service)

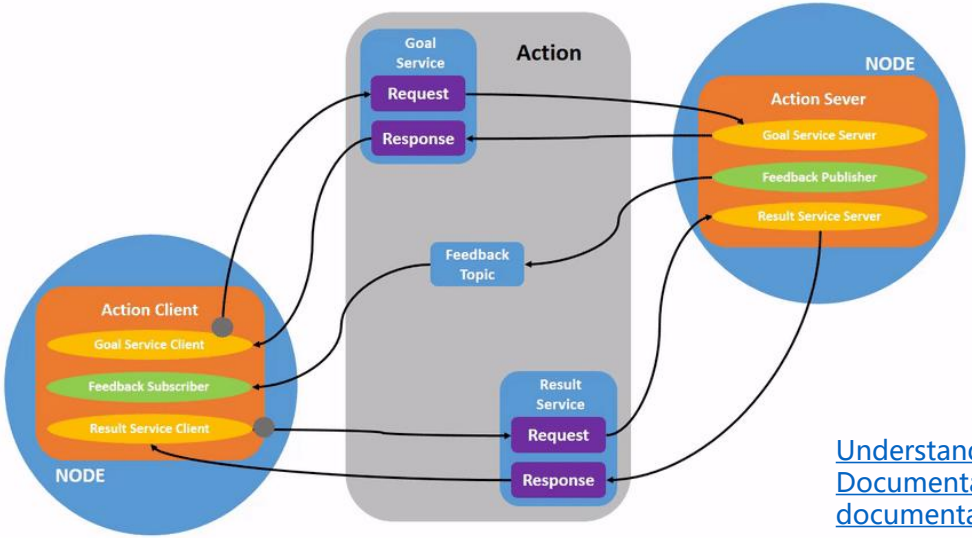
- 服务 (Service) —— 同步通信机制
- 使用客户端/服务器 (C/S) 模型，客户端发送请求数据，服务器完成处理后返回应答数据
- 使用编程语言无关的.srv文件定义请求和应答数据结构，编译过程中生成对应的代码文件
- 同一个服务，可以有多个客户端，但一个服务只能有一个服务器端。




[Understanding services — ROS 2 Documentation: Humble documentation]

# 3.4 动作（Action）

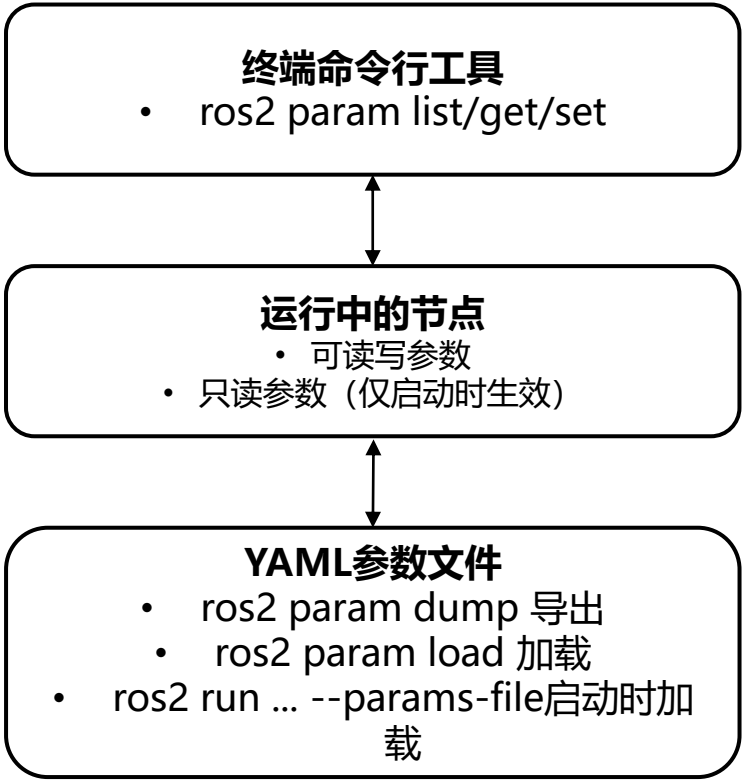
- Action专门面向长时间运行的机器人任务，基于 Topic 和 Service 底层实现，是服务的增强版
- Action 三大核心组成
  - Goal（目标）：客户端向服务器下发的任务指令（如“旋转到 1.57 弧度”、“导航到目标点”）
  - Feedback（实时反馈）：任务执行过程中，服务器周期性推送的进度信息（如“剩余旋转角度”、“当前距离目标点距离”）
  - Result（最终结果）：任务完成后返回的一次性最终输出（如“总旋转角度差”、“是否到达目标”）




[Understanding actions — ROS 2 Documentation: Humble documentation]

# 3.5 参数

- 参数（Parameter）是节点的配置项，你可以把参数理解成节点的配套参数系统，用于定义运行配置
- 节点支持存储整数、浮点数、布尔值、字符串、列表类型的参数
- 在 ROS2 中，每个节点独立维护自己的一套参数，互不干扰
- 你可以通过命令行工具随时读取、修改参数
- 也能将整套配置导出为 yaml 文件保存，下次启动节点直接加载复用

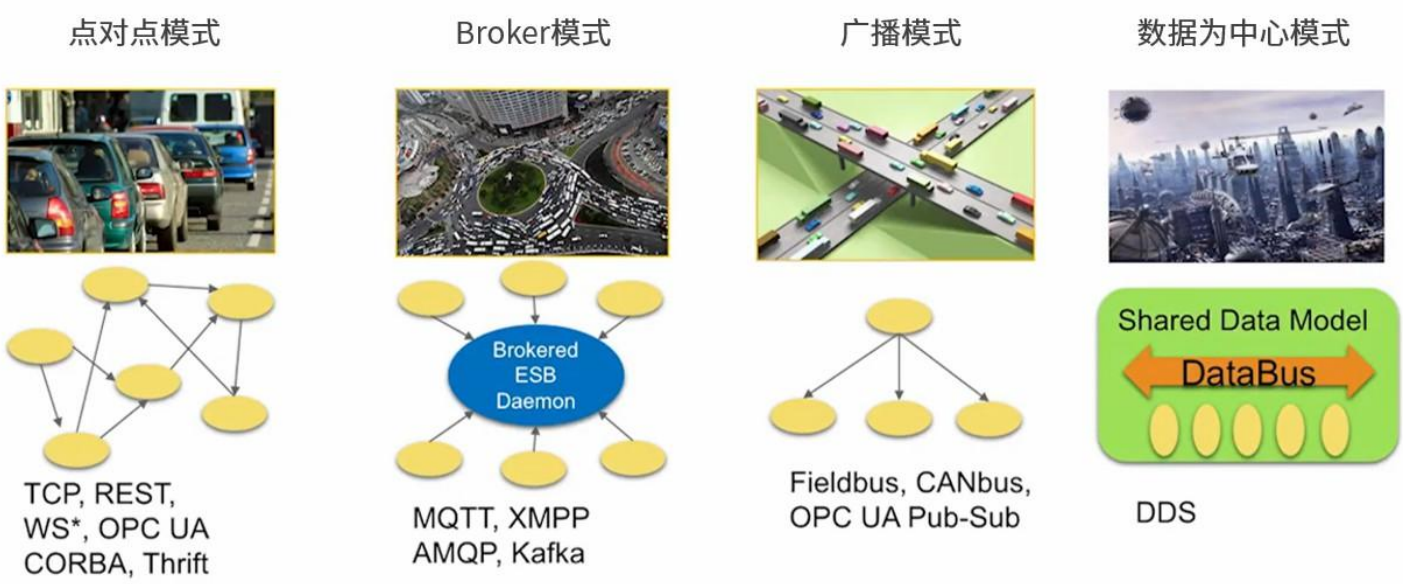



Understanding parameters — ROS 2 Documentation: Humble documentation

# 3.6 四种通信方式的对比

| 通信/配置方式 | 主要特点 | 典型适用场景 |
| --- | --- | --- |
| **Topic<br>话题** | ➢ 异步无阻塞通信，面向高频、单向、实时的数据流传输，是ROS中最基础的通信方式。 | ➢ 传感器数据（激光雷达、相机、IMU）、机器人状态广播、全局控制指令下发等场景。 |
| **Service<br>服务** | ➢ 同步阻塞通信，适用于一次性、简短的指令调用与状态查询，强调请求与结果的强关联。 | ➢ 触发机器人复位、生成地图实体、查询单次关节状态、参数获取确认等场景。 |
| **Action<br>动作** | ➢ 处理需要长时间执行的机器人任务，兼顾异步执行与过程可控性。 | ➢ 自主导航到目标点、机械臂关节运动规划、机器狗步态调整、长时间自主巡检等场景。 |
| **Parameter<br>参数** | ➢ 用于管理节点运行时的配置参数，实现程序逻辑与配置数据的分离。 | ➢ 设置机器人背景色、调整传感器量程阈值、配置算法参数、修改QoS策略等场景。 |

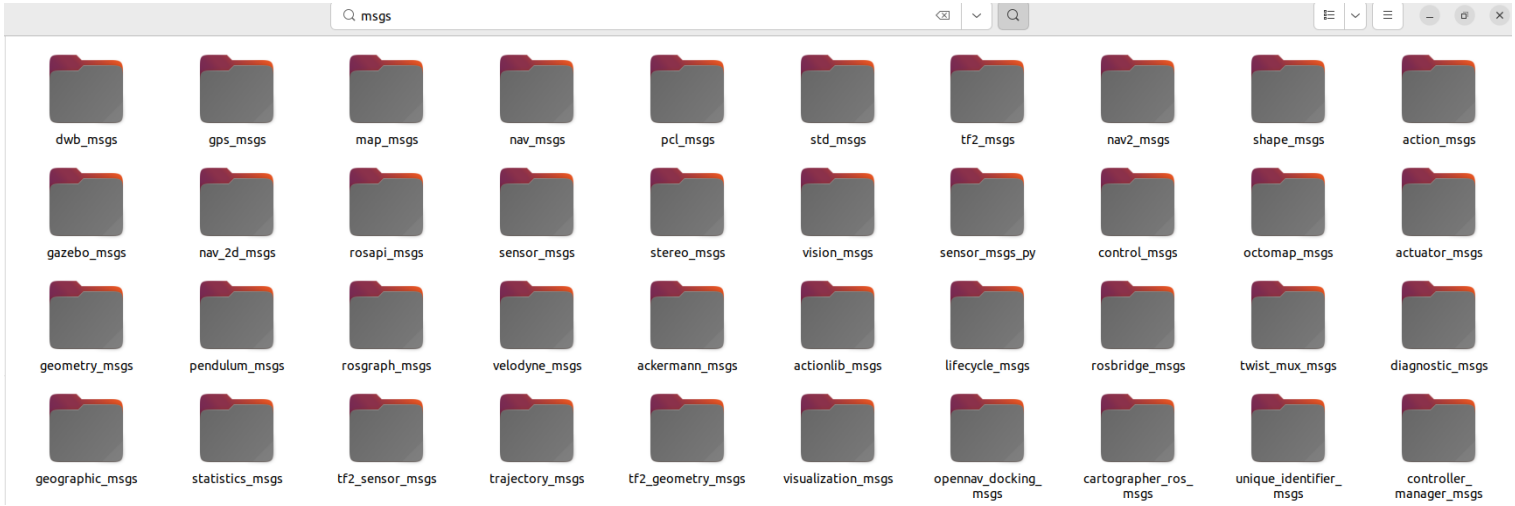
# 3.7 DDS



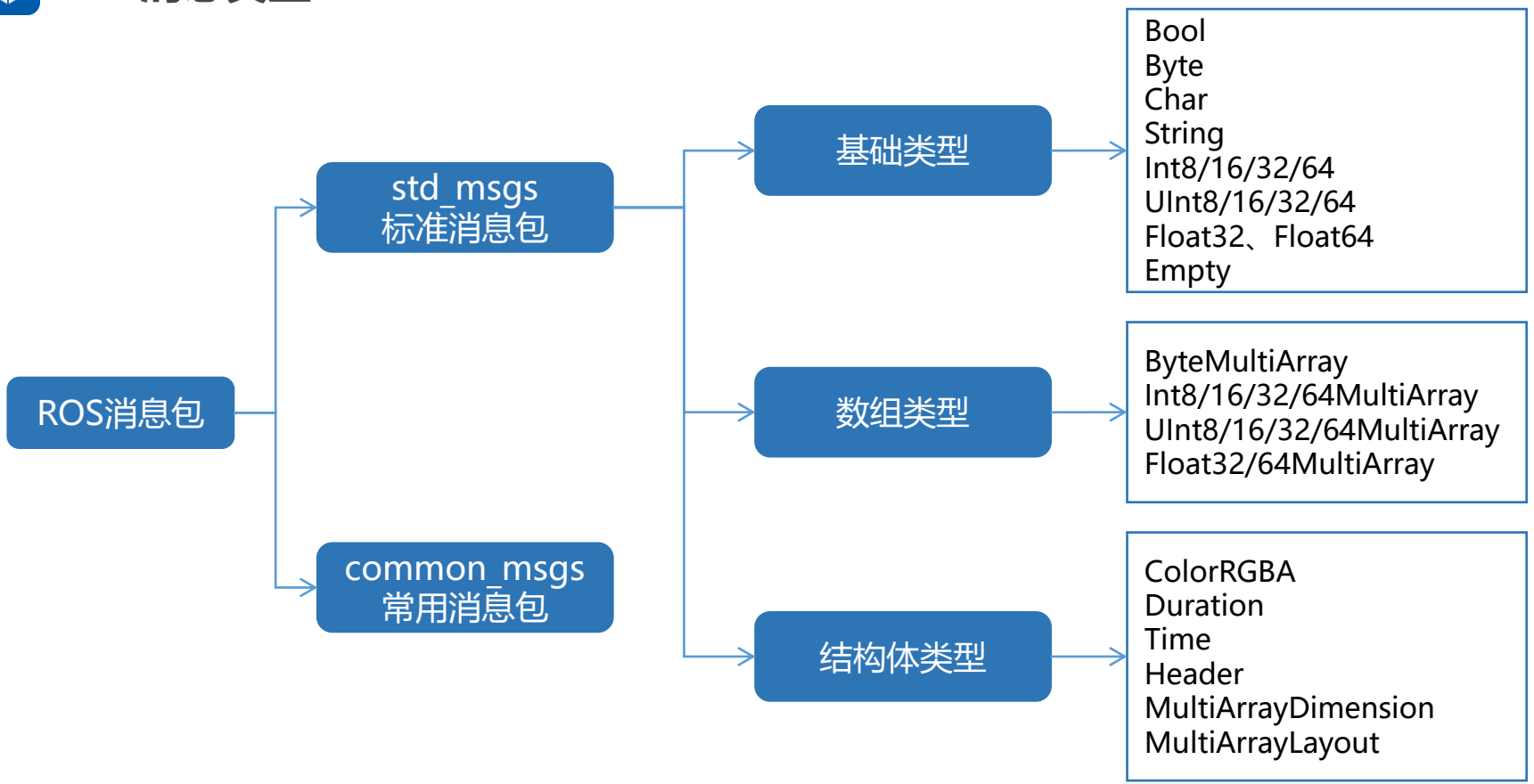

DDS的核心是通信，图为常用的四种通信模型。

# 3.8 消息类型

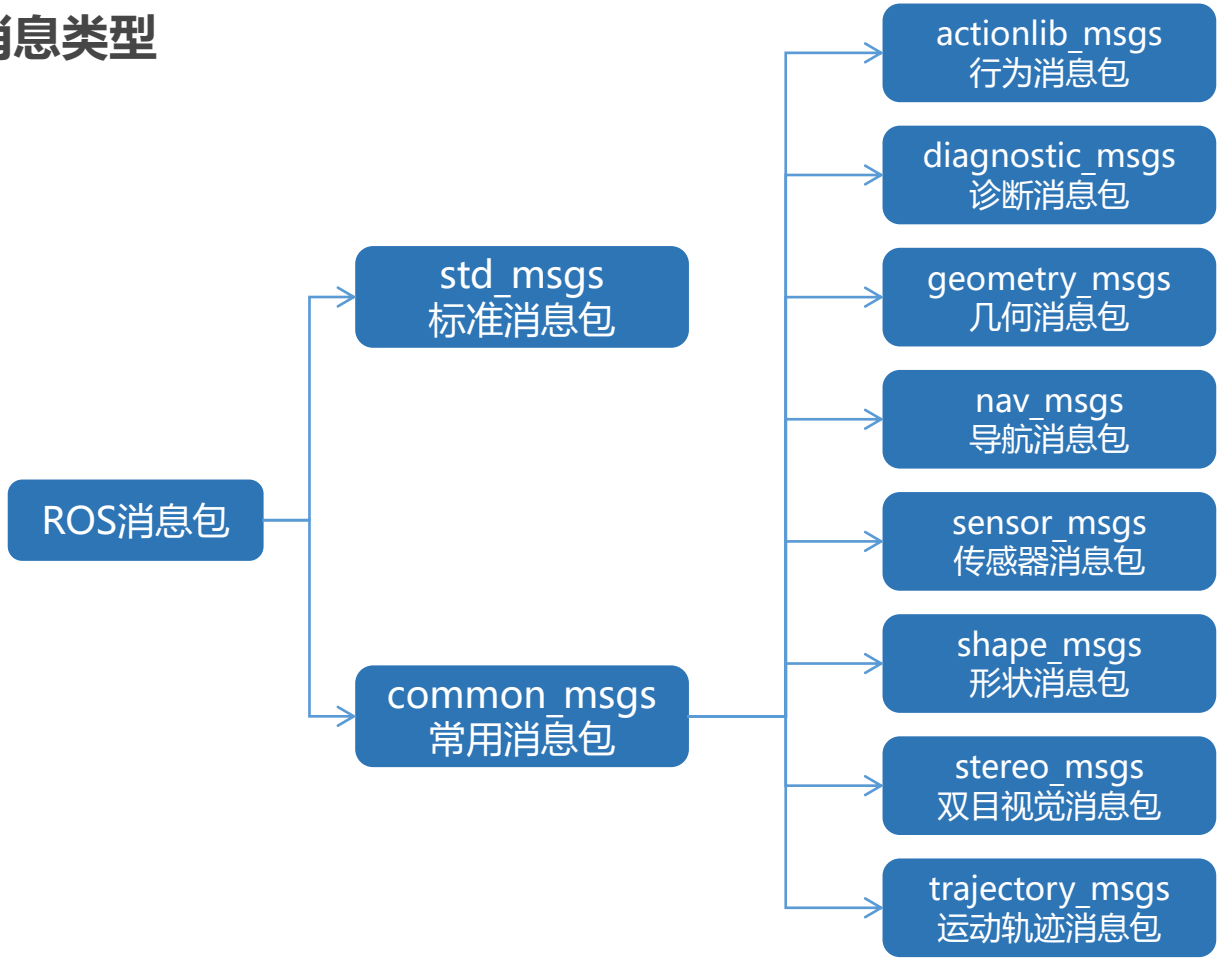
- ROS2 消息类型（Message Type）是节点之间通信的数据规范，规定了话题、服务、动作传输数据的字段名称、数据类型、结构
- 节点收发数据必须严格匹配同一种消息类型，否则无法通信
- 可以在 /opt/ros/humble/share 里查看ros2内置的消息类型包
- 也可以通过命令 ros2 interface list 查看更完整的内置消息类型
- 支持自定义消息类型



# 3.8 消息类型


ROS消息包
std_msgs
标准消息包
common_msgs
常用消息包
基础类型
数组类型
结构体类型
Bool
Byte
Char
String
Int8/16/32/64
UInt8/16/32/64
Float32、Float64
Empty
ByteMultiArray
Int8/16/32/64MultiArray
UInt8/16/32/64MultiArray
Float32/64MultiArray
ColorRGBA
Duration
Time
Header
MultiArrayDimension
MultiArrayLayout

# 3.8 消息类型


ROS消息包
std_msgs
标准消息包
common_msgs
常用消息包
actionlib_msgs
行为消息包
diagnostic_msgs
诊断消息包
geometry_msgs
几何消息包
nav_msgs
导航消息包
sensor_msgs
传感器消息包
shape_msgs
形状消息包
stereo_msgs
双目视觉消息包
trajectory_msgs
运动轨迹消息包

# 3.8 消息类型

common_msgs
常用消息包

geometry_msgs
几何消息包

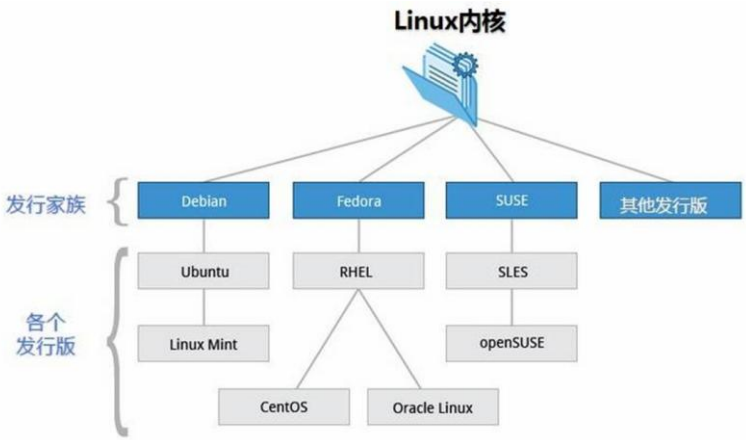
| | |
|---|---|
| 加速度 | Accel、AccelStamped、AccelWithCovariance、AccelWithCovarianceStamped |
| 惯量 | Inertia、InertiaStamped |
| 空间点 | Point、Point32、PointStamped |
| 多边形 | Polygon、PolygonStamped |
| 空间位置 | Pose、Pose2D、PoseArray、PoseStamped、PoseWithCovariance、PoseWithCovarianceStamped |
| 四元数 | Quaternion、QuaternionStamped |
| 空间变换 | Transform、TransformStamped |
| 空间方向 | Twist、TwistStamped、TwistWithCovariance、TwistWithCovarianceStamped |
| 三维矢量 | Vector3、Vector3Stamped |
| 扭矩 | Wrench、WrenchStamped |



# 4. ROS2安装方法

# 1.1 Linux系统简介

- Linux 内核最初只是由芬兰人林纳斯·托瓦兹（Linus Torvalds）在赫尔辛基大学上学时出于个人爱好而编写的。
- Linux的发行版就是将 Linux 内核与应用软件做一个打包。
- 目前市面上较知名的发行版有：Ubuntu、RedHat、CentOS、Debian、Fedora、SuSE、OpenSUSE、Arch Linux、SolusOS 等。













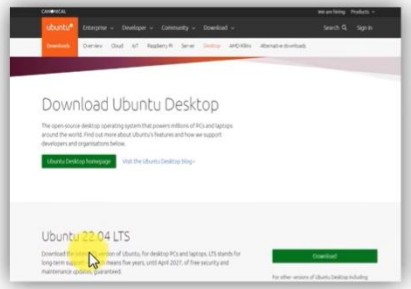

# 1.2 Ubuntu系统安装



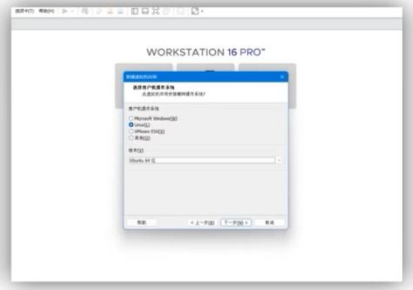
- Ubuntu是一个社区开发的基于Linux 的操作系统，适合笔记本、桌面计算机和服务器使用。它包含了常用的应用程序——网页浏览器、幻灯片演示、文档编辑和电子表格软件，即时通讯软件等等。
- 其名称来自非洲南部祖鲁语或豪萨语的“ubuntu"一词，意思是“人性” “我的存在是因为大家的存在"，是非洲传统的一种价值观。
- 2004年10月，Ubuntu 的第一个版本正式发布。此后Ubuntu每六个月发布一个新版本，用户可以免费升级到最新版本。Ubuntu的命名遵从“Y.MM(开发代号)"格式，Y代表年份，MM代表月份。
- 每一个普通版本都将被支持18个月，长期支持版(LongTerm Support，LTS)通常支持5年时间。。



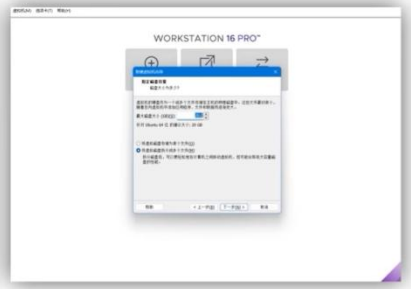
# 1.3 Ubuntu虚拟机安装



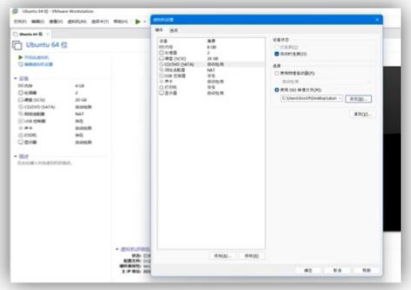

下载镜像



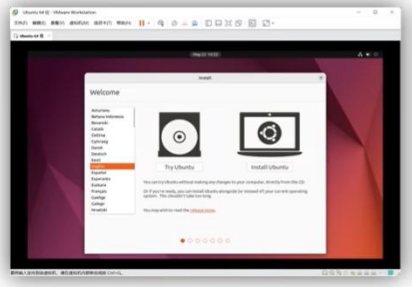

创建系统



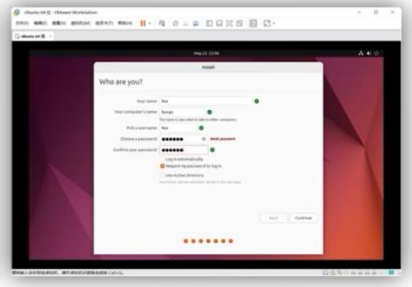

硬件配置



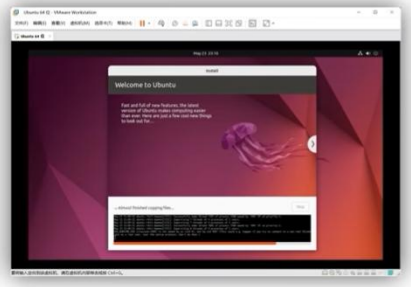
配置镜像



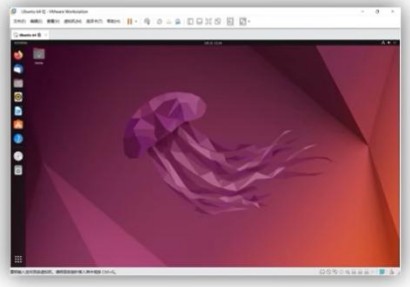

系统安装




创建用户




等待安装

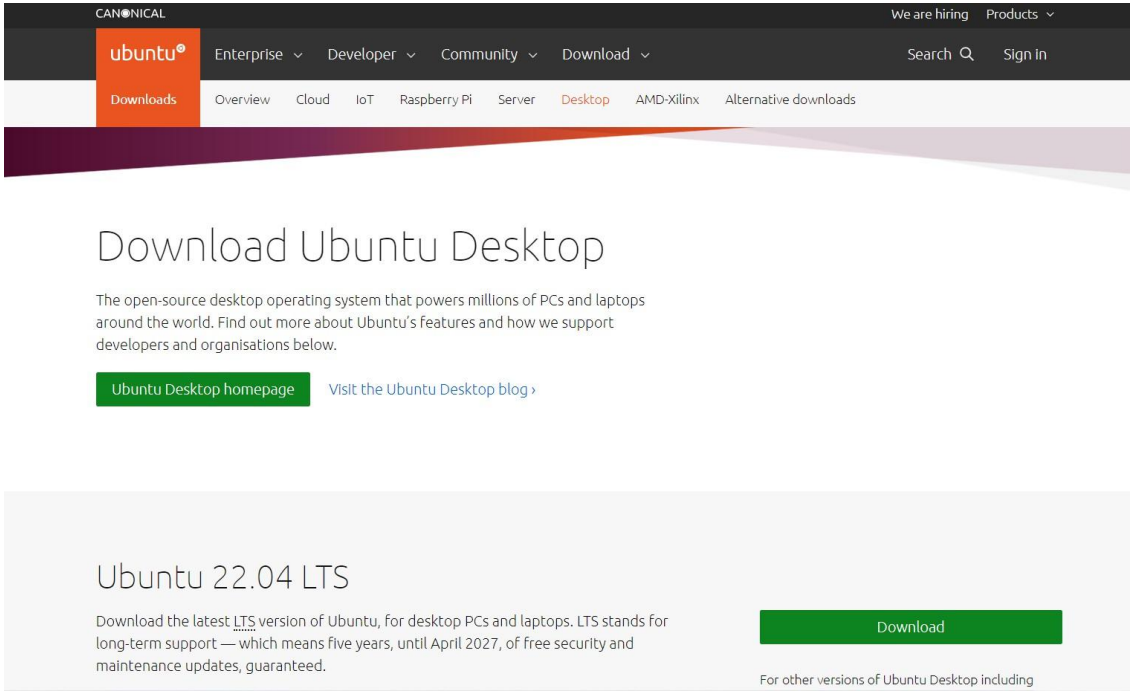


进入系统

- Ubuntu镜像下载地址: https://ubuntu.com/download/desktop
- VMware Workstation Pro: https://www.vmware.com/products/workstation-pro/workstation-pro-evaluation.html

# 1.3 Ubuntu虚拟机安装

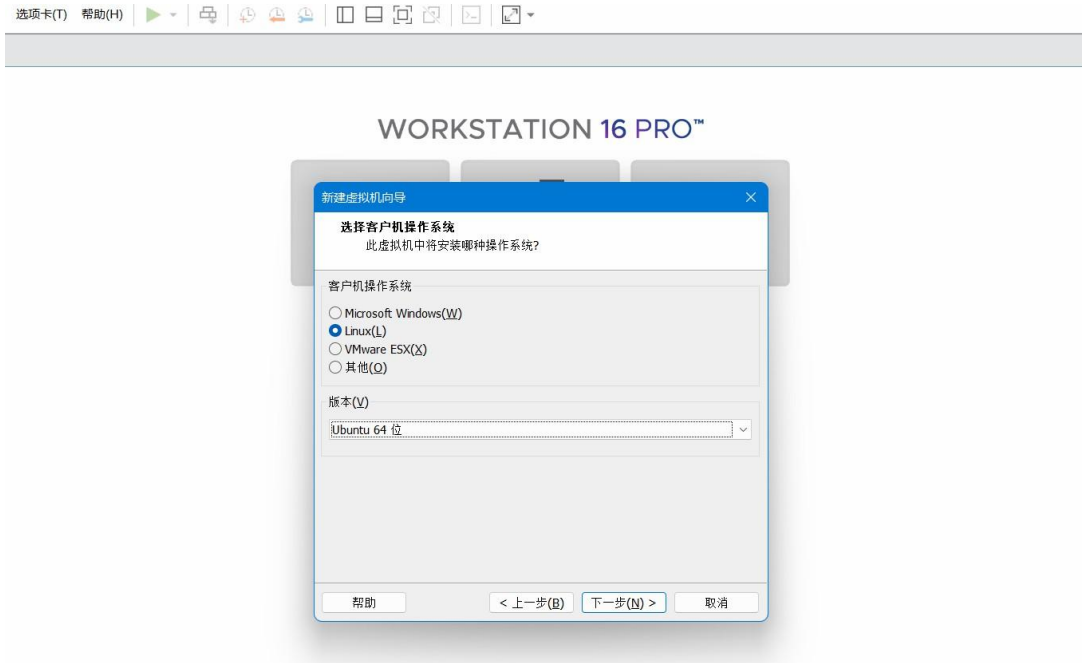
1. 下载系统镜像




- 下载链接：https://ubuntu.com/download/desktop

# 1.3 Ubuntu虚拟机安装

2. 在虚拟机中创建系统



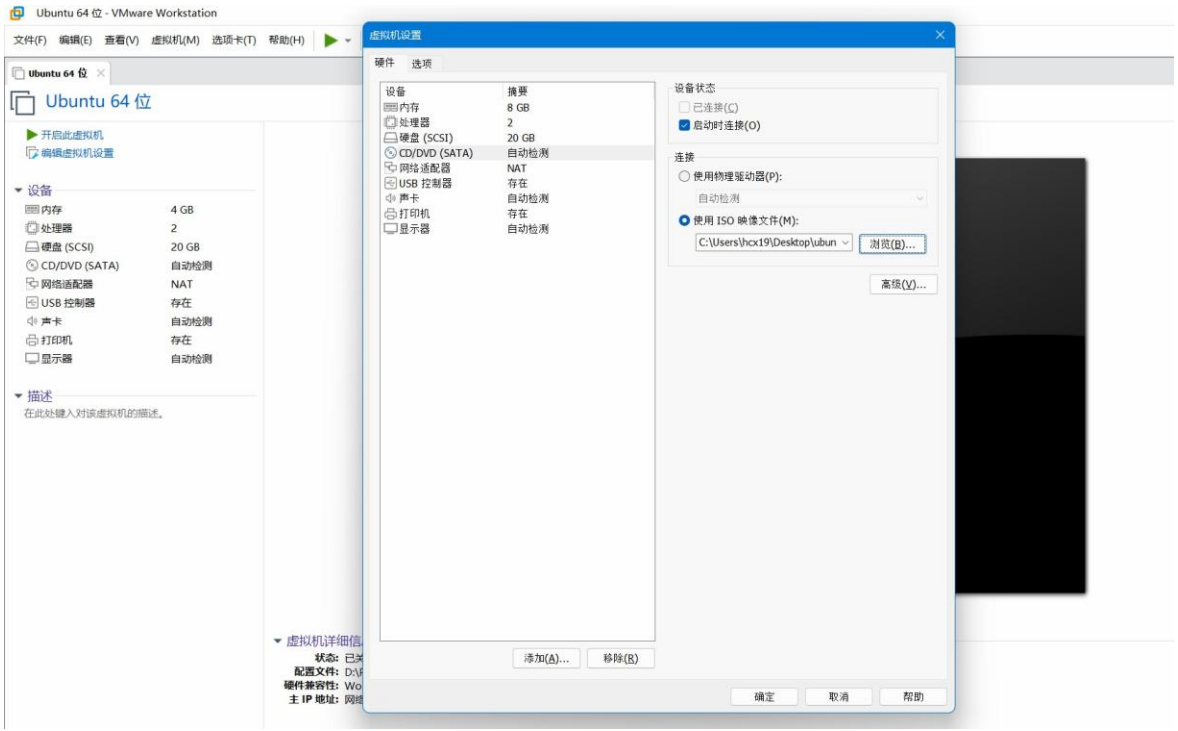
# 1.3 Ubuntu虚拟机安装

3.设置虚拟机硬盘大小



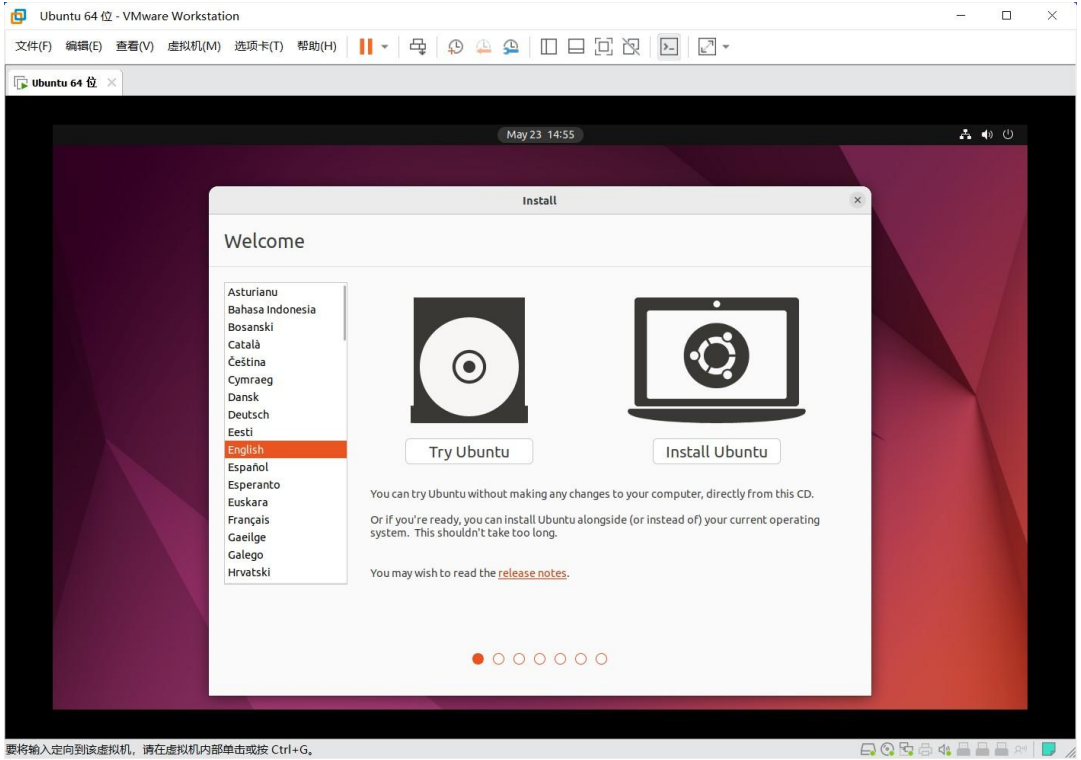
# 1.3 Ubuntu虚拟机安装

4.设置Ubuntu镜像路径



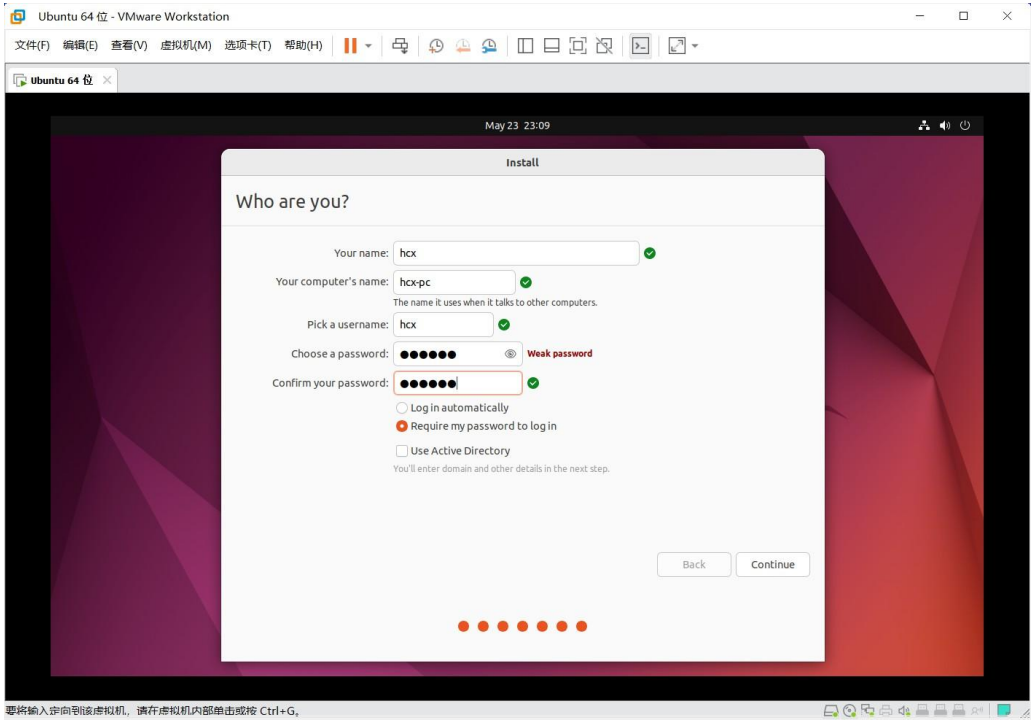
# 1.3 Ubuntu虚拟机安装

5.启动虚拟机



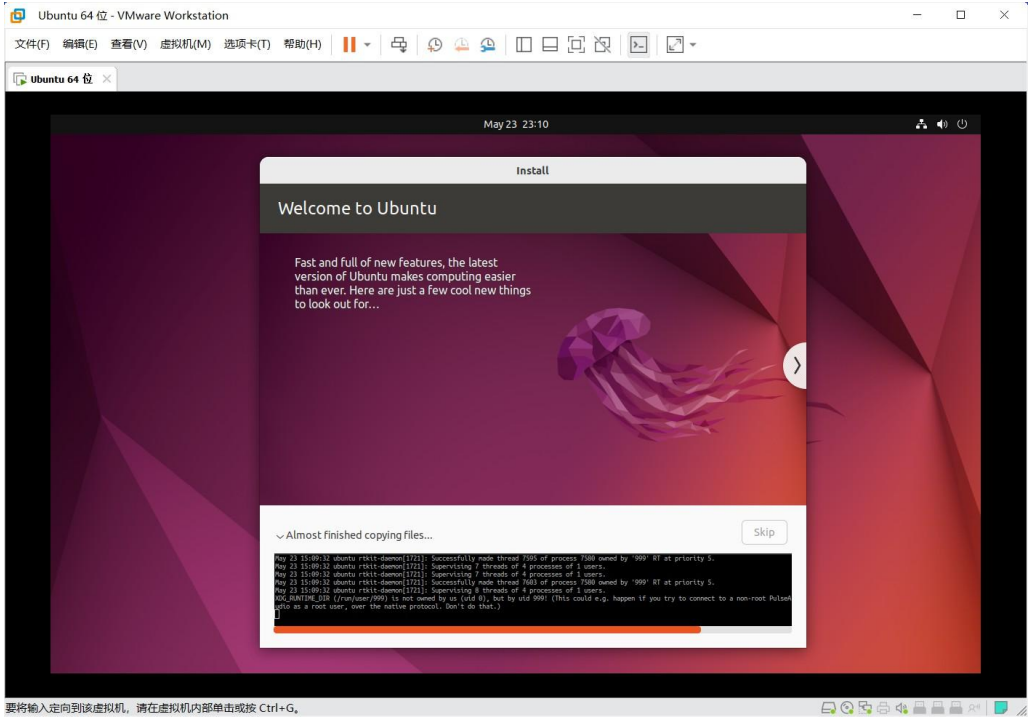
# 1.3 Ubuntu虚拟机安装

6.设置用户名和密码



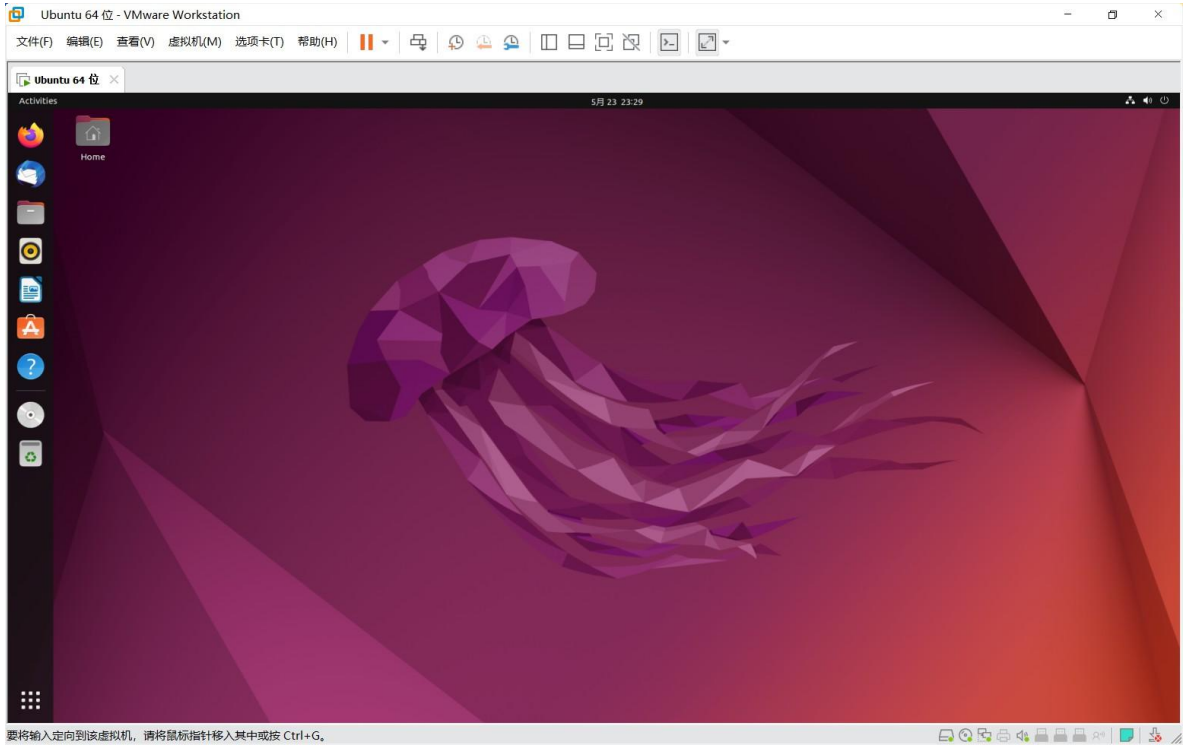
# 1.3 Ubuntu虚拟机安装

7.等待系统安装



# 1.3 Ubuntu虚拟机安装

8.完成安装



# 1.4 安装ROS2

## 设置编码

```
$ sudo apt update && sudo apt install locales
$ sudo locale-gen en_US en_US.UTF-8
$ sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
$ export LANG=en_US.UTF-8
```

## 添加源

```
$ sudo apt update && sudo apt install curl gnupg lsb-release
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(source /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

## 安装ROS2

```
$ sudo apt update
$ sudo apt upgrade
$ sudo apt install ros-humble-desktop
```

## 设置环境变量

```
$ source /opt/ros/humble/setup.bash
$ echo " source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

# 1.5 ROS2示例测试

## 命令行示例

$ ros2 run demo_nodes_cpp talker

```
hcx@hcx-pc:~$ ros2 run demo_nodes_cpp talker
[INFO] [1649324903.872856816] [talker]: Publishing: 'Hello World: 1'
[INFO] [1649324904.872275317] [talker]: Publishing: 'Hello World: 2'
[INFO] [1649324905.872913303] [talker]: Publishing: 'Hello World: 3'
[INFO] [1649324906.872675069] [talker]: Publishing: 'Hello World: 4'
[INFO] [1649324907.872411691] [talker]: Publishing: 'Hello World: 5'
[INFO] [1649324908.872845391] [talker]: Publishing: 'Hello World: 6'
[INFO] [1649324909.872517248] [talker]: Publishing: 'Hello World: 7'
[INFO] [1649324910.872843282] [talker]: Publishing: 'Hello World: 8'
[INFO] [1649324911.872164476] [talker]: Publishing: 'Hello World: 9'
[INFO] [1649324912.873188896] [talker]: Publishing: 'Hello World: 10'
[INFO] [1649324913.872963853] [talker]: Publishing: 'Hello World: 11'
[INFO] [1649324914.872992945] [talker]: Publishing: 'Hello World: 12'
[INFO] [1649324915.872893756] [talker]: Publishing: 'Hello World: 13'
[INFO] [1649324916.872297067] [talker]: Publishing: 'Hello World: 14'
[INFO] [1649324917.872913526] [talker]: Publishing: 'Hello World: 15'
[INFO] [1649324918.872234402] [talker]: Publishing: 'Hello World: 16'
```

$ ros2 run demo_nodes_py listener


```
hcx@hcx-pc:~$ ros2 run demo_nodes_cpp talker
[INFO] [1649324903.872856816] [talker]: Publishing: 'Hello World: 1'
[INFO] [1649324904.872275317] [talker]: Publishing: 'Hello World: 2'
[INFO] [1649324905.872913303] [talker]: Publishing: 'Hello World: 3'
[INFO] [1649324906.872675069] [talker]: Publishing: 'Hello World: 4'
[INFO] [1649324907.872411691] [talker]: Publishing: 'Hello World: 5'
[INFO] [1649324908.872845391] [talker]: Publishing: 'Hello World: 6'
[INFO] [1649324909.872517248] [talker]: Publishing: 'Hello World: 7'
[INFO] [1649324910.872843282] [talker]: Publishing: 'Hello World: 8'
[INFO] [1649324911.872164476] [talker]: Publishing: 'Hello World: 9'
[INFO] [1649324912.873188896] [talker]: Publishing: 'Hello World: 10'
[INFO] [1649324913.872963853] [talker]: Publishing: 'Hello World: 11'
[INFO] [1649324914.872992945] [talker]: Publishing: 'Hello World: 12'
[INFO] [1649324915.872893756] [talker]: Publishing: 'Hello World: 13'
[INFO] [1649324916.872297067] [talker]: Publishing: 'Hello World: 14'
[INFO] [1649324917.872913526] [talker]: Publishing: 'Hello World: 15'
[INFO] [1649324918.872234402] [talker]: Publishing: 'Hello World: 16'
```

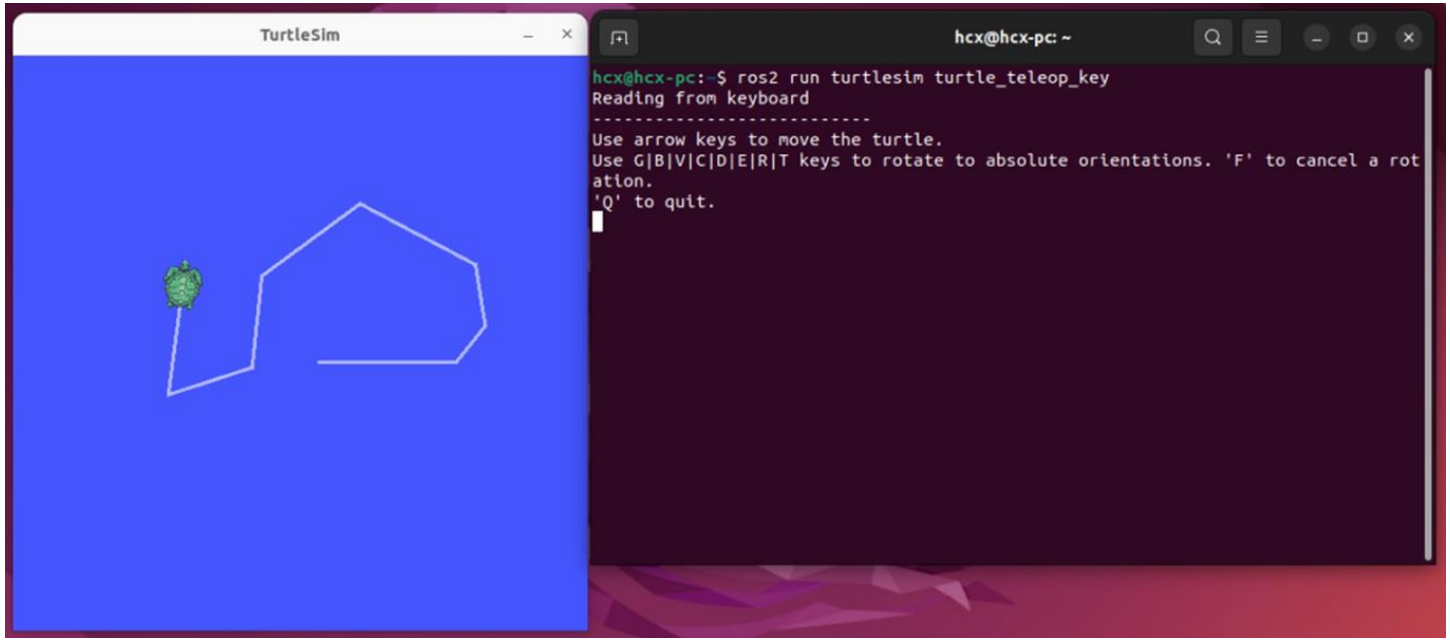
# 1.5 ROS2示例测试

## 小海龟仿真示例

启动两个终端，分别运行如下指令：

$ ros2 run turtlesim turtlesim_node

$ ros2 run turtlesim turtle_teleop_key





# 5. ROS2体验：小海龟仿真

# 4. ROS2体验：小海龟仿真

- **功能包安装**

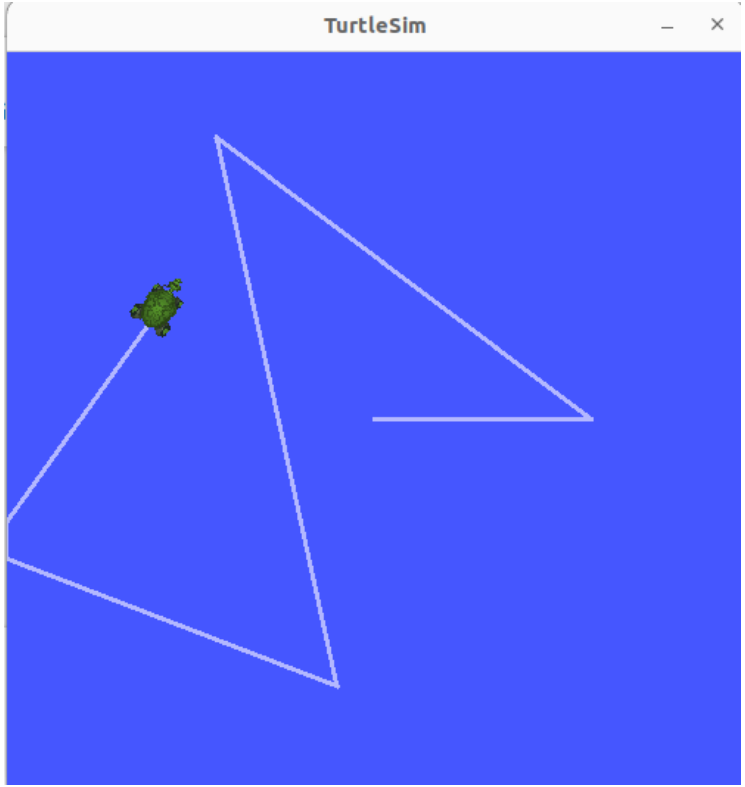
```
$ sudo apt install ros-humble-turtlesim
```

- **启动小海龟仿真节点**

```
$ ros2 run turtlesim turtlesim_node
```

- **启动小海龟控制节点**

```
$ ros2 run turtlesim turtle_teleop_key
```

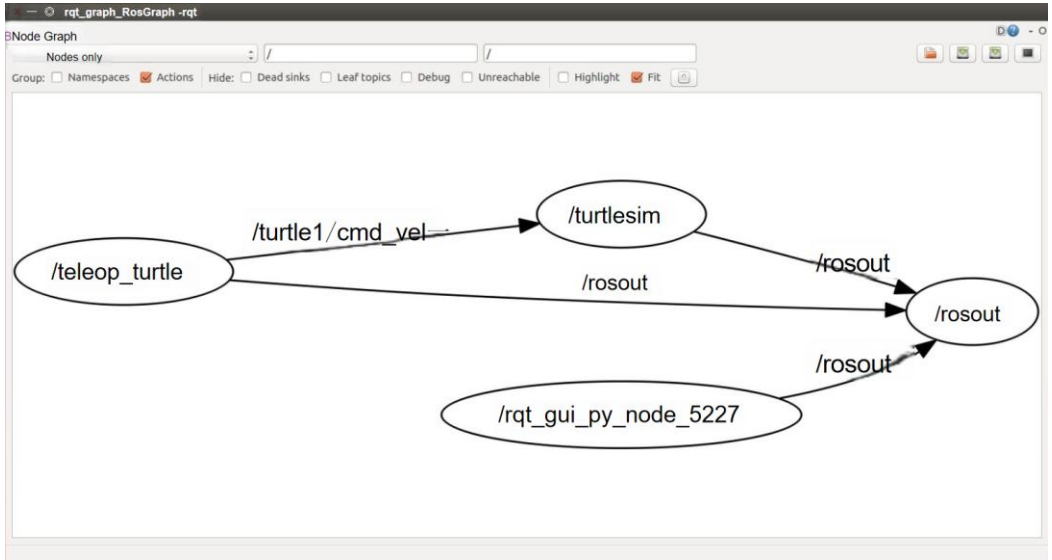


# 4. ROS2体验：小海龟仿真

## ➢ 查看相关信息

```
$ ros2 node list
$ ros2 topic list
$ ros2 service list
$ ros2 action list

$ ros2 node info <node>
$ ros2 topic echo <topic>
$ ros2 topic hz <topic>
$ ros2 topic info <topic>
```




## ➢ 通过rqt_graph查看节点间的联系

```
$ rqt_graph
```

# 4. ROS2体验：小海龟仿真

➢ **查看相关信息**

```
$ ros2 node list
$ ros2 topic list
$ ros2 service list
$ ros2 action list

$ ros2 node info <node>
$ ros2 topic echo <topic>
$ ros2 topic hz <topic>
$ ros2 topic info <topic>
```

➢ **通过rqt_graph查看节点间的联系**

```
$ rqt_graph
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 node list
/teleop_turtle
/turtlesim
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 node info /turtlesim
/turtlesim
  Subscribers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /turtle1/cmd_vel: geometry_msgs/msg/Twist
  Publishers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
    /turtle1/color_sensor: turtlesim/msg/Color
    /turtle1/pose: turtlesim/msg/Pose
  Service Servers:
    /clear: std_srvs/srv/Empty
    /kill: turtlesim/srv/Kill
    /reset: std_srvs/srv/Empty
    /spawn: turtlesim/srv/Spawn
    /turtle1/set_pen: turtlesim/srv/SetPen
    /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
    /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
    /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
    /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
    /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
    /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:
    /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
  Action Clients:
```

# 4. ROS2体验：小海龟仿真

## ➢ 查看相关信息

$ ros2 node list
$ ros2 topic list
$ ros2 service list
$ ros2 action list

$ ros2 node info <node>
$ ros2 topic info <topic>
$ ros2 interface show <msg>
$ ros2 topic echo <topic>
$ ros2 topic hz <topic>

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 topic info /turtle1/pose
Type: turtlesim/msg/Pose
Publisher count: 1
Subscription count: 0
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
```

# 4. ROS2体验：小海龟仿真

## ➢ 查看相关信息

$ ros2 node list
$ ros2 topic list
$ ros2 service list
$ ros2 action list

$ ros2 node info <node>
$ ros2 topic info <topic>
$ ros2 interface show <msg>
$ ros2 topic echo <topic>
$ ros2 topic hz <topic>

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 topic echo /turtle1/pose --once
x: 4.136275768280029
y: 4.849863529205322
theta: -1.339185357093811
linear_velocity: 0.0
angular_velocity: 0.0
---
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 topic hz  /turtle1/pose
average rate: 62.443
        min: 0.015s max: 0.017s std dev: 0.00054s window: 64
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 action list
/turtle1/rotate_absolute
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~$ ros2 service list
/clear
/kill
/reset
/spawn
/teleop_turtle/describe_parameters
/teleop_turtle/get_parameter_types
/teleop_turtle/get_parameters
/teleop_turtle/list_parameters
/teleop_turtle/set_parameters
/teleop_turtle/set_parameters_atomically
/turtle1/set_pen
/turtle1/teleport_absolute
```

# 感谢倾听！

Thanks for Listening

